

ASSIGNMENT - 01

CSE 430(SOFTWARE TESTING)

FROM

JANNATUL FERDAUS OISHI

ID-2022-3-60-216

TO

Shartaz Sajid Nahid

Lecturer

Department of Computer Science & Engineering

DATE:4 February, 2026

Task-1(Calculator)

```

package-info.java package-info.java Calculator1.java Calculator1TestCase.java module-info.java
import Implementation.CalculatorI;

public class Calculator1TestCase {
    Calculator1 calculator;

    @Before
    public void setup() {
        calculator = new Calculator1();
    }

    @Test
    public void testAdd() {
        assertEquals(10, calculator.add(5, 5));
    }

    @Test
    public void testSubtraction() {
        assertEquals(10, calculator.subtraction(5, -5));
    }

    @Test
    public void testPower() {
        assertEquals(8, calculator.power(2, 3));
    }

    @Test
    public void testMod() {
        assertEquals(1, calculator.modulus(5, 2));
    }

    @Test
    public void testPowerZeroExponent() {
        assertEquals(1, calculator.power(5, 0));
    }

    @Test
    public void testModBySameNum() {
        assertEquals(0, calculator.modulus(5, 5));
    }

    @Test
    public void testDivideByZero() {
        assertEquals(10, calculator.division(10, 0));
    }

    @Test
    public void testModulusByZero() {
        assertEquals(0, calculator.modulus(5, 0));
    }

    @Test
    public void testNegativeExponent() {
        assertEquals(0.5, calculator.power(2, -1));
    }
}

```

Method	Input Values	Expected output	Result
add	(5, 5)	10	pass
Substraction	(5, -5)	10	pass
power	(2, 3)	8	pass
modulus	(5,2)	1	pass
power	(5,0)	1	pass
modulus	(5,5)	0	pass
division	(10, 0)	IllegalArgumentExcepti on	pass
modulus	(5, 0)	IllegalArgumentExcepti	Pass

		on	
power	(5, -2)	IllegalArgumentException on	Pass

Calculator Reflection

This calculator testing task helped me understand how unit testing works in a practical way. I learned how to test basic arithmetic operations and also how to handle special cases like division by zero and negative values. Using assertions such as **assertEquals** and **assertThrows** made it easier to check whether the results and errors were correct. I also realized why using **@Before** is important, as it makes sure each test starts fresh. Overall, this exercise improved my confidence in writing unit tests and helped me create cleaner, more reliable code.

Task-1(Account Validation)

The screenshot shows an IDE with a Java file named `BankTestCase` and its test results. The code is as follows:

```

1 import static org.junit.Assert.assertEquals;
2 import static org.junit.Assert.assertNotEquals;
3 import static org.junit.Assert.assertTrue;
4
5 import org.junit.Before;
6 import org.junit.Test;
7
8 import Implementation.BankAccount;
9
10 public class BankTestCase {
11     BankAccount account;
12
13     @Before
14     public void setUp() {
15         account = new BankAccount();
16     }
17
18     @Test
19     public void TestValidDeposit() {
20         account.deposit(500);
21         assertEquals(500, account.getBalance(), 0.001);
22     }
23
24     @Test
25     public void TestInvalidDeposit() {
26         assertEquals(IllegalArgumentException.class, () -> {
27             account.deposit(-100);
28         });
29     }
30
31     @Test
32     public void TestValidWithdraw() {
33         account.deposit(400);
34         account.withdraw(100);
35         assertEquals(300, account.getBalance(), 0.001);
36     }
37
38     @Test
39     public void TestInvalidWithdraw() {
40         account.deposit(500);
41         assertEquals(IllegalArgumentException.class, () -> {
42             account.withdraw(600);
43         });
44     }
45
46     @Test
47     public void TestActiveAccount() {
48         account.deposit(150);
49         assertTrue(account.isActive());
50     }
51 }

```

The test results on the right show that all tests passed:

- TestValidDeposit() : void
- TestInvalidDeposit() : void
- TestValidWithdraw() : void
- TestInvalidWithdraw() : void
- TestActiveAccount() : void

Method	Input Values	Expected output	Result
deposit	500	500.0	Pass
deposit	-100	Throws IllegalArgumentException	Pass
withdraw	Deposit 400, Withdraw 100	300.0	Pass
withdraw	Deposit 500, Withdraw 700	Throws IllegalArgumentException	Pass
isActive()	Deposit 150, Withdraw 100	Returns false	Pass
isActive()	Deposit 150	Returns true	Pass

Reflection

This testing exercise helped me understand how to write better and more reliable unit tests. I learned that it is important to use the correct assertion methods, especially when comparing floating-point values using assertEquals with a delta. Writing test cases for exceptions also helped me check how the program behaves with wrong inputs. Using @Before made sure that each test started with a fresh object, which avoided unexpected errors. I also became more careful when calculating expected results. Overall, the tests now check normal, boundary, and error cases, making the program stronger and easier to maintain.

Task-3(String Utility Testing)

```

import Implementation.StringAnalyzer;

public class StringTest {
    StringAnalyzer s;

    @Before
    public void setUp() {
        s = new StringAnalyzer();
    }

    @Test
    //null input should return false
    public void testIsPalindromeNull() {
        assertFalse(s.isPalindrome(null));
    }

    @Test
    //Palindrome: empty string should return false
    public void testIsPalindromeEmpty() {
        assertFalse(s.isPalindrome(""));
    }

    @Test
    //Palindrome: case-insensitive check
    public void testIsPalindromeCaseInsensitive() {
        assertTrue(s.isPalindrome("madam"));
    }

    @Test
    //vowel count
    public void testVowelCountNormal() {
        assertEquals(s.countVowels("hello"));
    }

    @Test
    //no vowels
    public void testVowelCountVowels() {
        assertEquals(0, s.countVowels("bcdfg"));
    }

    @Test
    //null input
    public void testVowelCountNull() {
        assertEquals(0, s.countVowels(null));
    }

    @Test
    //valid anagram
    public void testValidAnagram() {
        assertTrue(s.isAnagram("listen", "silent"));
    }

    @Test
    //different words should return false
    public void testInvalidAnagram() {
        assertFalse(s.isAnagram("hello", "world"));
    }
}

```

Method	Input Values	Expected output	Result
isPalindrome()	null	false	Pass
isPalindrome()	" "	false	Pass
isPalindrome()	"Madam"	true	Pass
countVowels()	"Hello"	2	Pass
countVowels()	"bcdfg"	0	Pass
isAnagram()	"Listen", "Silent"	true	Pass
isAnagram()	"Hello", "World"	false	Pass
isAnagram()	" ", " "	false	Pass
countVowels()	null	0	pass

Reflection

While working on this task, I learned how to implement and test different string-processing methods using JUnit. Creating test cases for null, empty, and case-insensitive inputs helped me realize how important it is to consider edge cases, not just normal inputs. I also understood how unit testing helps catch mistakes early and gives confidence that the code is working as

expected. If I had more time, I would improve the program by handling punctuation and special characters and by adding more test cases for unusual inputs. Overall, this exercise helped me think more carefully about writing clean code and testing it properly.